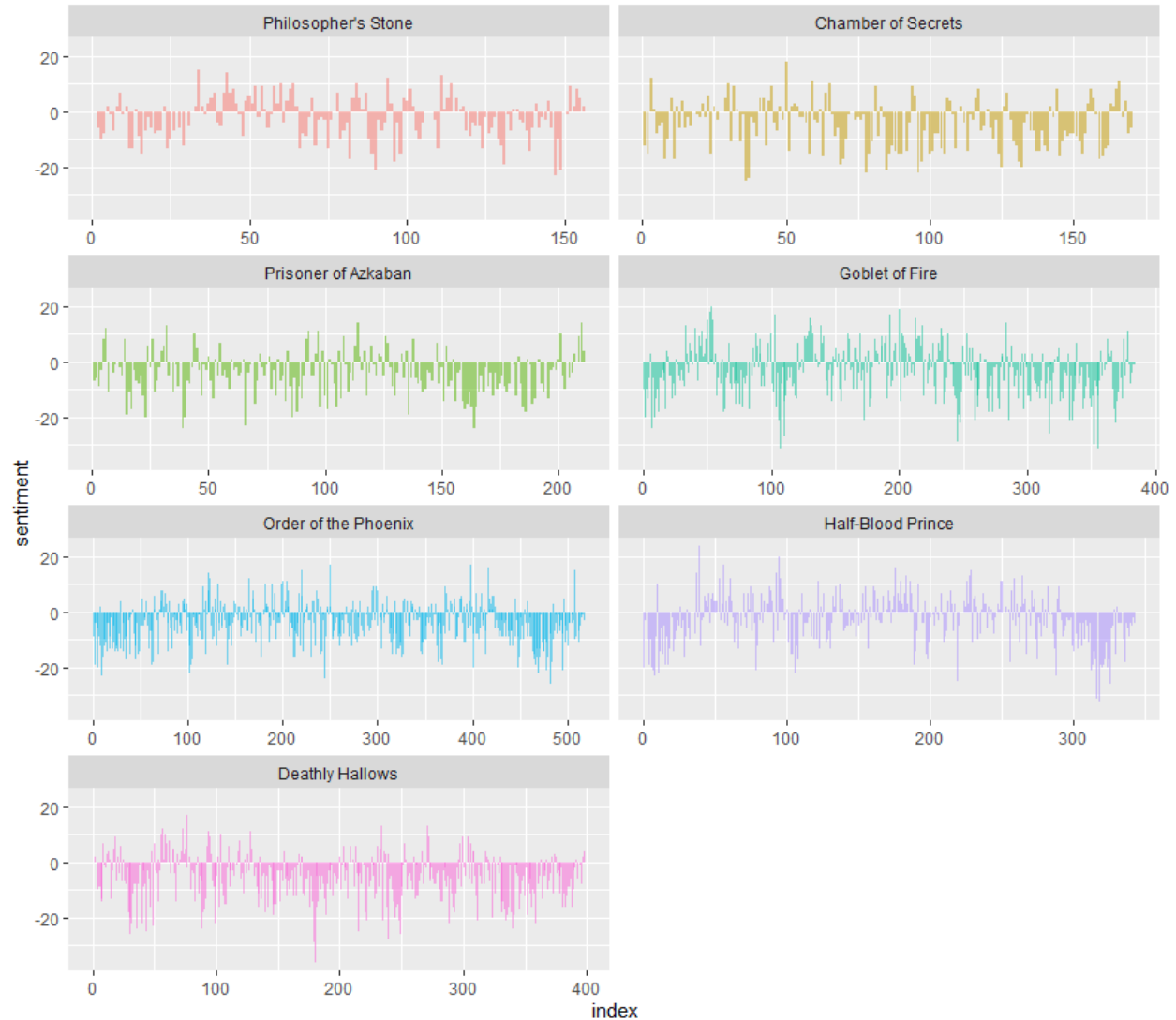


I picked R tutorial #2 because I am a huge fan of Harry Potter, and I appreciate how easy this tutorial was to comprehend. I would like to note I did have to tweak the code just a bit to get it to work, I left comments if changes were made to the tutorials code.

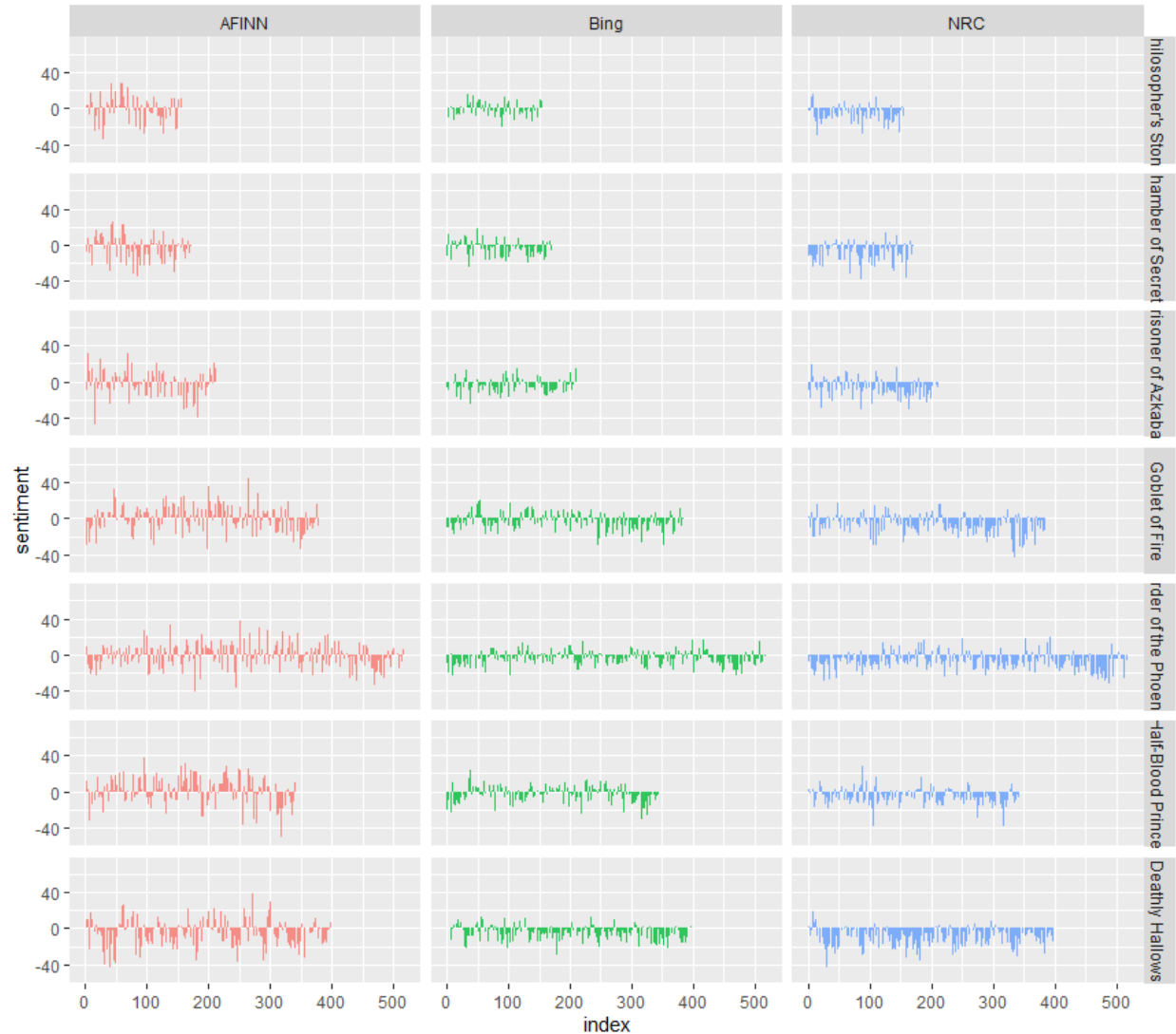
Code/Output:

```
> # NRC sentiment data set
> series %>%
+   right_join(get_sentiments("nrc")) %>%
+   filter(!is.na(sentiment)) %>%
+   count(sentiment, sort = TRUE)
Joining with `by = join_by(word)`
# A tibble: 10 × 2
  sentiment      n
  <chr>      <int>
1 negative    55096
2 positive    37767
3 sadness     34883
4 anger       32747
5 trust       23160
6 fear        21536
7 anticipation 20629
8 joy         13804
9 disgust     12861
10 surprise    12818
```

Interpretation: The overall sentiment distribution shows that negative words appear more frequently than positive words across the entire Harry Potter series. Negative emotions such as sadness, fear, and anger appear more frequently, which suggests the books plots contain substantial conflict and emotional tension. Positive emotions such as trust, anticipation, and joy also appear quite frequently, which suggests themes of hope and growth to balance out the tone of the series. It is a conflict-driven book series rather than being purely positive however.

Code/Output:

Interpretation: The plots show net sentiment, positive to negative, per 500 word segment for each book. The sentiment fluctuates throughout each book however it is noticeably more negative as the series goes on, especially *Deathly Hallows*. *Philosopher's Stone* is the most neutral out of all the books.

Code/Output:

Interpretation: All 3 sentiment methods reveal similar overall trends, which suggests that major emotional highs and lows are occurring in roughly the same places regardless of lexicon. AFINN produces the more larger peaks and valleys, since it assigns a weighted score. Bing and NRC have less extreme values due to classifying words as negative and positive.

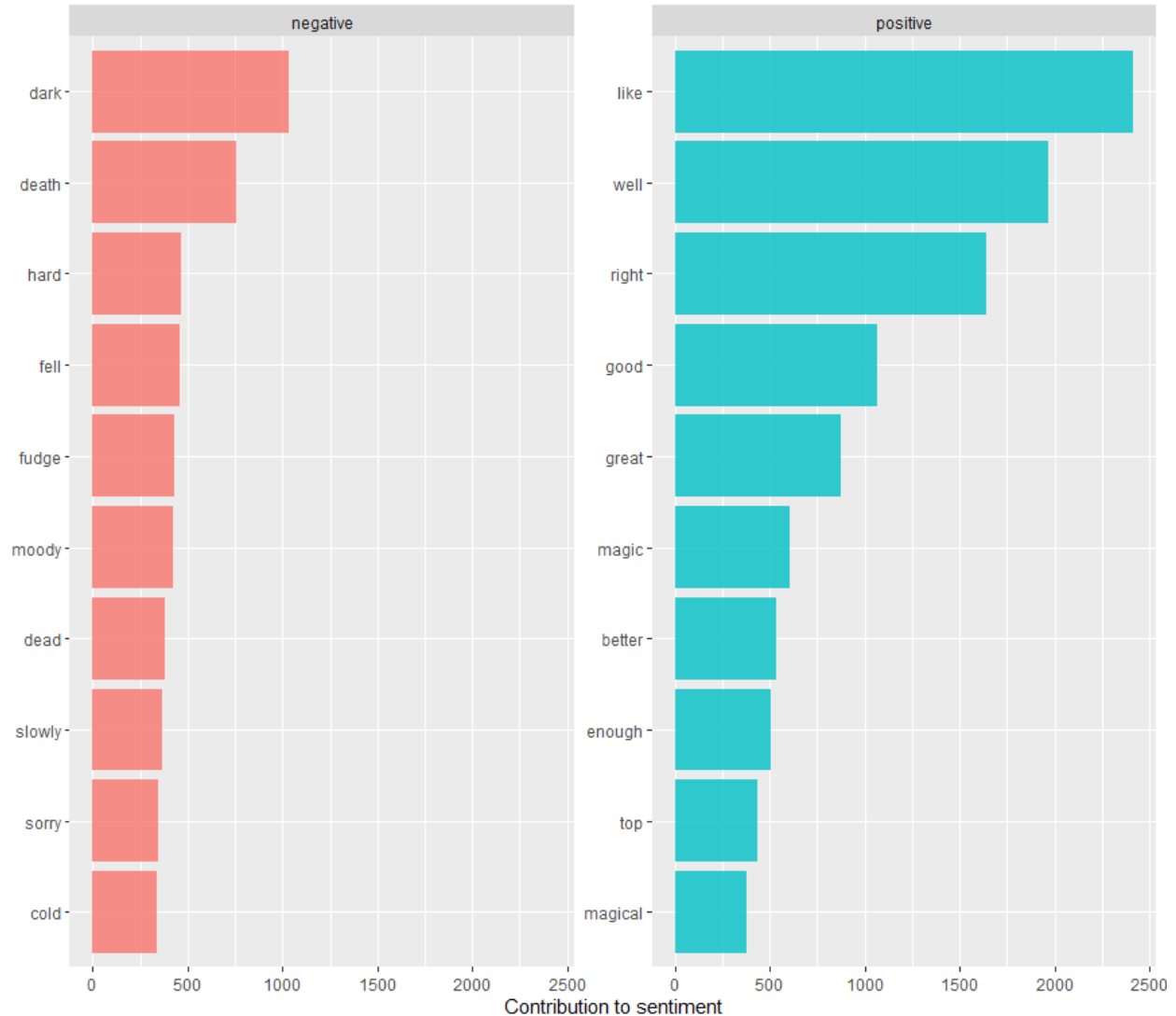
Code/Output:

```

> # Common Sentiment Words
> bing_word_counts <- series %>%
+   inner_join(get_sentiments("bing")) %>%
+   count(word, sentiment, sort = TRUE) %>%
+   ungroup()
Joining with `by = join_by(word)`
warning message:
In inner_join(., get_sentiments("bing")) :
  Detected an unexpected many-to-many relationship between `x` and
  `y`.
i Row 41435 of `x` matches multiple rows in `y`.
i Row 2698 of `y` matches multiple rows in `x`.
i If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning.>
> bing_word_counts
# A tibble: 3,313 × 3
   word      sentiment      n
  <chr>    <chr>      <int>
1 like    positive    2416
2 well    positive    1969
3 right   positive    1643
4 good    positive    1065
5 dark    negative    1034
6 great   positive     877
7 death   negative     757
8 magic    positive     606
9 better   positive     533
10 enough positive     509
# i 3,303 more rows
# i Use `print(n = ...)` to see more rows

```

Interpretation: The most frequent positive words are “like”, “well”, “right”, “good”, and “great”. These words suggest that much of the dialogue/narration includes expressions of encouragement and affirmation. The word “magic” appears among the frequent positive words which reinforces the central theme of the series (wizards). The most frequent negative words are “dark” and “death”. These words suggest that the narrative addresses danger, mortality, threatening situations, and symbolic uses of darkness throughout the series.

Code/Output:

Interpretation: The bar chart shows the top 10 most frequent positive and negative words used in the series. Just like in the last interpretation, positive words such as “like”, “well”, “right”, “good”, “great”, and “magic” appear most frequently. The negative words that appear most frequently are “dark”, “death” (just like in the last interpretation), “hard”, “fell”, and “fudge”. This reinforces the idea that it is a conflict-driven book series rather than being purely positive.

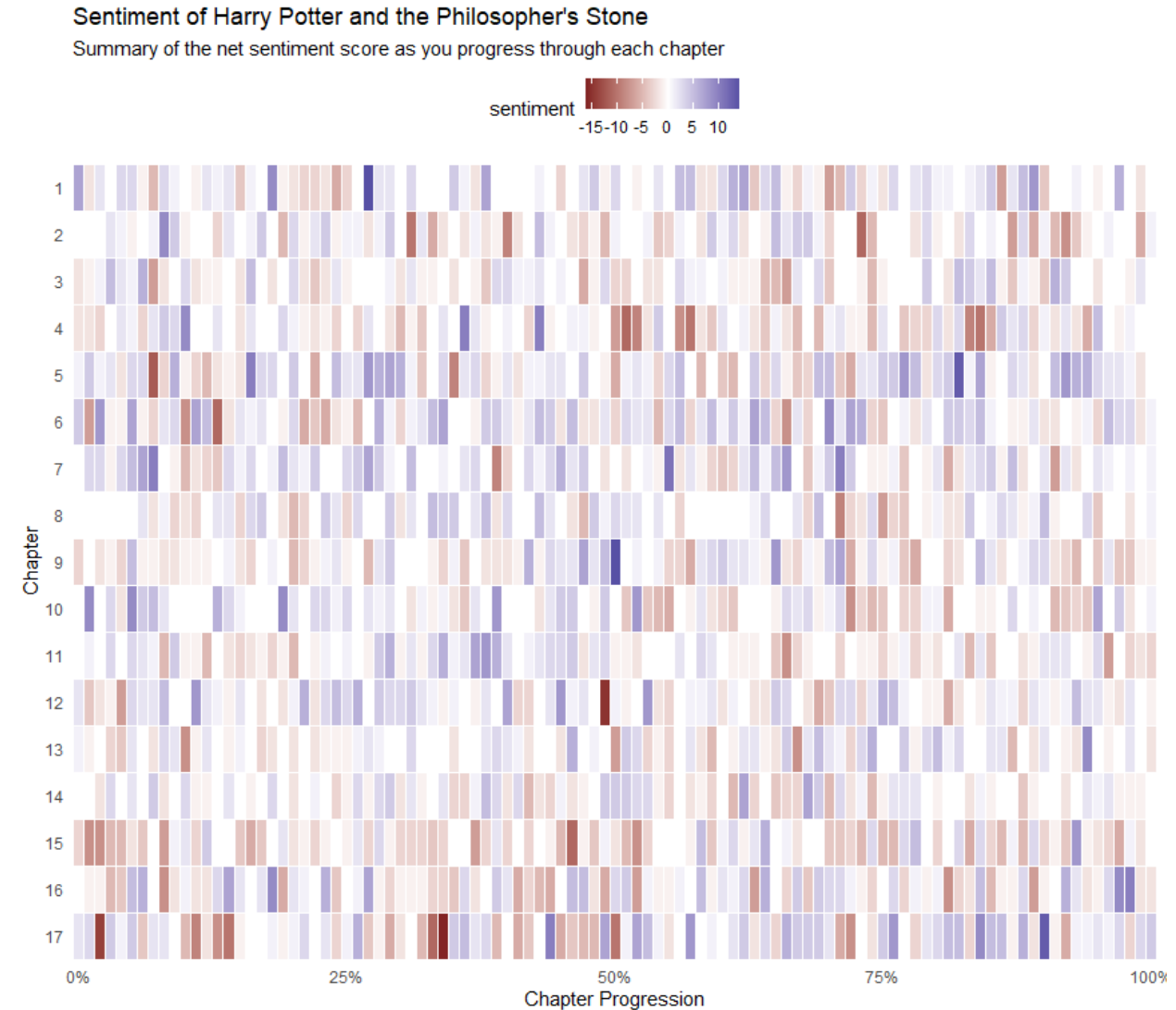
Code/Output:

```

> # sentiments by chapter
> book_sent <- ps_sentences %>%
+   unnest_tokens(word, sentence) %>%           # had tokenize first, kept get
ting an error in 'group_by()'
+   group_by(chapter) %>%
+   mutate(sentence_num = row_number(),
+     index = round(sentence_num / n(), 2)) %>%
+   inner_join(get_sentiments("afinn")) %>%
+   group_by(chapter, index) %>%
+   summarise(sentiment = sum(value, na.rm = TRUE), .groups = "drop") %>% # a
gain had to change 'score' to 'value'
+   arrange(desc(sentiment))
Joining with `by = join_by(word)`
>
> book_sent
# A tibble: 1,474 × 3
   chapter index sentiment
   <int> <dbl>    <dbl>
1       1  0.27      14
2       9  0.5       14
3       5  0.82      13
4      17  0.9       13
5       7  0.55      11
6       4  0.36      10
7       4  0.43      10
8       5  0.16      10
9       7  0.07      10
10      7  0.71      10
# i 1,464 more rows
# i Use `print(n = ...)` to see more rows

```

Interpretation: This table shows that the strongest positive sentiments in book 1 the Philosopher's Stone are chapters 1, 9, 5, and 17 (the last chapter). Chapter 9 has very strong positive moments because it is Harry's first flight lessons, also when he joins the Gryffindor Quidditch team, and when he goes on a midnight adventure. Chapter 7 appears quite frequently in the tibble, which is when the students get sorted into their houses by the sorting hat, which has its positive moments.

Code/Output:

Interpretation: The heatmap shows the net sentiment per chapter in the Philosopher's Stone. Red means positive, blue means negative, and white means neutral. The sentiment clearly fluctuates per chapter, many chapters begin relatively neutral and then shift towards a stronger emotional tone near the middle and end. Later chapters display stronger contrasts between positive and negative, reflecting tension near the climax of the story. The final chapter shows more positive sentiment towards the beginning and middle, with more neutral sentiment towards the end, suggesting a resolution and victory in the final chapter. The patterns throughout the book clearly show a rising tension, climax, and eventful resolution.

Full R Code:

```
1 # Download package from Github
2 if (packageversion("devtools") < 1.6) {
3   install.packages("devtools")
4 }
5
6 devtools::install_github("bradleyboehmke/harrypotter")
7
8 # Load libraries
9 library(tidyverse)      # data manipulation & plotting
10 library(stringr)        # text cleaning and regular expressions
11 library(tidytext)        # provides additional text mining functions
12 library(harrypotter)     # provides the first seven novels of the Harry Potter series
13
14 |
15
16 # First two chapters
17 philosophers_stone[1:2]
18
19 # Sentiment data sets
20 sentiments
21
22 # to see the individual lexicons try
23 get_sentiments("afinn")
24 get_sentiments("bing")
25 get_sentiments("nrc")
26
27 # Basic Sentiment Analysis
28 titles <- c("Philosopher's Stone", "Chamber of Secrets", "Prisoner of Azkaban",
29            "Goblet of Fire", "Order of the Phoenix", "Half-Blood Prince",
30            "Deathly Hallows")
31
32 books <- list(philosophers_stone, chamber_of_secrets, prisoner_of_azkaban,
33             goblet_of_fire, order_of_the_phoenix, half_blood_prince,
34             deathly_hallows)
35
36 series <- tibble()
37
38 for(i in seq_along(titles)) {
39   clean <- tibble(chapter = seq_along(books[[i]]),
40                  text = books[[i]]) %>%
41     unnest_tokens(word, text) %>%
42     mutate(book = titles[i]) %>%
43     select(book, everything())
44
45   series <- rbind(series, clean)
46 }
47
48
49 # set factor to keep books in order of publication
50 series$book <- factor(series$book, levels = rev(titles))
51
52 series
53
54 # NRC sentiment data set
55 series %>%
56   right_join(get_sentiments("nrc")) %>%
57   filter(!is.na(sentiment)) %>%
58   count(sentiment, sort = TRUE)
```



```

60 # Index that breaks up each book by 500 words
61 # Assesses the positive vs negative sentiment of each word
62 # Counts how many positive and negative words there are for every two pages
63 # Calculate nt sentiment
64 # Plots data
65 series %>%
66   group_by(book) %>%
67   mutate(word_count = 1:n(),
68          index = word_count %/% 500 + 1) %>%
69   inner_join(get_sentiments("bing")) %>%
70   count(book, index = index, sentiment) %>%
71   ungroup() %>%
72   spread(sentiment, n, fill = 0) %>%
73   mutate(sentiment = positive - negative,
74          book = factor(book, levels = titles)) %>%
75   ggplot(aes(index, sentiment, fill = book)) +
76   geom_bar(alpha = 0.5, stat = "identity", show.legend = FALSE) +
77   facet_wrap(~ book, ncol = 2, scales = "free_x")
78
79 # Comparing sentiments
80 afinn <- series %>%
81   group_by(book) %>%
82   mutate(word_count = 1:n(),
83          index = word_count %/% 500 + 1) %>%
84   inner_join(get_sentiments("afinn")) %>%
85   group_by(book, index) %>%
86   summarise(sentiment = sum(value)) %>% # had to change 'score' to 'value' from the tutorial for code to work
87   mutate(method = "AFINN")
88
89 bing_and_nrc <- bind_rows(series %>%
90   group_by(book) %>%
91   mutate(word_count = 1:n(),
92          index = word_count %/% 500 + 1) %>%
93   inner_join(get_sentiments("bing")) %>%
94   mutate(method = "Bing"),
95   series %>%
96   group_by(book) %>%
97   mutate(word_count = 1:n(),
98          index = word_count %/% 500 + 1) %>%
99   inner_join(get_sentiments("nrc")) %>%
100   filter(sentiment %in% c("positive", "negative")) %>%
101   mutate(method = "NRC")) %>%
102   count(book, method, index = index, sentiment) %>%
103   ungroup() %>%
104   spread(sentiment, n, fill = 0) %>%
105   mutate(sentiment = positive - negative) %>%
106   select(book, index, method, sentiment)
107
108 # Bind the net sentiments together and plot
109 bind_rows(afinn,
110   bing_and_nrc) %>%
111   ungroup() %>%
112   mutate(book = factor(book, levels = titles)) %>%
113   ggplot(aes(index, sentiment, fill = method)) +
114   geom_bar(alpha = 0.8, stat = "identity", show.legend = FALSE) +
115   facet_grid(book ~ method)

```

```

117 # Common Sentiment words
118 bing_word_counts <- series %>%
119   inner_join(get_sentiments("bing")) %>%
120   count(word, sentiment, sort = TRUE) %>%
121   ungroup()
122
123 bing_word_counts
124
125 # view top n words for each sentiment visually
126 bing_word_counts %>%
127   group_by(sentiment) %>%
128   top_n(10) %>%
129   ggplot(aes(reorder(word, n), n, fill = sentiment)) +
130   geom_bar(alpha = 0.8, stat = "identity", show.legend = FALSE) +
131   facet_wrap(~sentiment, scales = "free_y") +
132   labs(y = "contribution to sentiment", x = NULL) +
133   coord_flip()
134
135 # Sentiment analysis with larger units
136 tibble(text = philosophers_stone) %>%
137   unnest_tokens(sentence, text, token = "sentences")
138
139 # breaking up philosophers_stone text by chapter and sentence
140 ps_sentences <- tibble(chapter = 1:length(philosophers_stone),
141   text = philosophers_stone) %>%
142   unnest_tokens(sentence, text, token = "sentences")
143
144 # sentiments by chapter
145 book_sent <- ps_sentences %>%
146   unnest_tokens(word, sentence) %>% # had tokenize first, kept getting an error in 'group_by()'
147   group_by(chapter) %>%
148   mutate(sentence_num = row_number(),
149     index = round(sentence_num / n(), 2)) %>%
150   inner_join(get_sentiments("afinn")) %>%
151   group_by(chapter, index) %>%
152   summarise(sentiment = sum(value, na.rm = TRUE), .groups = "drop") %>% # again had to change 'score' to 'value'
153   arrange(desc(sentiment))
154
155 book_sent
156
157 # visualization (heatmap) of sentiment as we progress through each chapter
158 ggplot(book_sent, aes(index, factor(chapter, levels = sort(unique(chapter), decreasing = TRUE)), fill = sentiment)) +
159   geom_tile(color = "white") +
160   scale_fill_gradient2() +
161   scale_x_continuous(labels = scales::percent, expand = c(0, 0)) +
162   scale_y_discrete(expand = c(0, 0)) +
163   labs(x = "Chapter Progression", y = "Chapter") +
164   ggtitle("Sentiment of Harry Potter and the Philosopher's Stone",
165     subtitle = "Summary of the net sentiment score as you progress through each chapter") +
166   theme_minimal() +
167   theme(panel.grid.major = element_blank(),
168     panel.grid.minor = element_blank(),
169     legend.position = "top")

```

10:28 (Top Level) ±